

Ejercicios de Programación 1 en Java - Paso a Paso

Materia: Programación 1 - DGETP

1. VARIABLES

Ejercicio 1.1 - Datos Personales

Objetivo: Aprender a declarar y usar diferentes tipos de variables.

Paso a paso:

1. Crear una nueva clase llamada `DatosPersonales`
2. En el método `main`, declarar las siguientes variables:

```
java  
  
String nombre;  
int edad;  
double altura;  
boolean esEstudiante;
```

3. Asignar valores a cada variable usando el operador `=`
4. Usar `System.out.println()` para mostrar cada variable con un mensaje descriptivo
5. Compilar y ejecutar el programa

Ejemplo de salida esperada:

```
Nombre: Juan Pérez  
Edad: 18 años  
Altura: 1.75 metros  
Es estudiante: true
```

Ejercicio 1.2 - Calculadora de Área

Paso a paso:

1. Crear clase `CalculadoraArea`
2. Declarar variable `double radio = 5.0;`
3. Crear variable `double area` y calcular usando: `Math.PI * radio * radio`
4. Mostrar el resultado con formato: "El área del círculo con radio X es: Y"
5. Probar con diferentes valores de radio

Ejercicio 1.3 - Conversión de Temperaturas

Paso a paso:

1. Crear clase `ConversionTemperatura`
2. Declarar `double celsius = 25.0;`
3. Calcular fahrenheit: `double fahrenheit = (celsius * 9.0/5.0) + 32;`
4. Mostrar ambas temperaturas
5. Verificar el resultado manualmente

Ejercicio 1.4 - Tienda de Productos

Paso a paso:

1. Crear clase `TiendaProductos`
2. Declarar variables: nombre del producto, precio, cantidad, disponible
3. Calcular valor total: `double valorTotal = precio * cantidad;`
4. Mostrar resumen del producto
5. Cambiar valores y verificar cálculos

Ejercicio 1.5 - Datos del Estudiante

Paso a paso:

1. Crear clase `DatosEstudiante`
2. Declarar: cedula (String), carrera (String), semestre (int), promedio (double)
3. Asignar valores representativos
4. Crear un método para mostrar resumen académico
5. Formatear la salida de manera profesional

DESAFÍO VARIABLES: Crear un programa que simule una factura de compra con: cliente, fecha, 3 productos (nombre, precio, cantidad), calcular subtotal, IVA (22%), y total final. Usar al menos 10 variables de diferentes tipos.

2. ESTRUCTURAS DE CONTROL

Ejercicio 2.1 - Clasificación de Edades

Paso a paso:

1. Crear clase `ClasificacionEdades`
2. Importar Scanner: `import java.util.Scanner;`

3. Crear objeto Scanner: `Scanner scanner = new Scanner(System.in);`

4. Solicitar edad: `System.out.print("Ingrese su edad: ");`

5. Leer edad: `int edad = scanner.nextInt();`

6. Usar if-else if para clasificar:

```
java

if (edad <= 12) {
    System.out.println("Niño");
} else if (edad <= 17) {
    System.out.println("Adolescente");
} // continuar...
```

7. Cerrar scanner: `scanner.close();`

Ejercicio 2.2 - Calculadora Básica

Paso a paso:

1. Crear clase `CalculadoraBasica`

2. Solicitar dos números usando Scanner

3. Solicitar operación (char): `char operacion = scanner.next().charAt(0);`

4. Usar switch:

```
java

switch (operacion) {
    case '+':
        resultado = num1 + num2;
        break;
    // otros casos...
}
```

5. Mostrar resultado o mensaje de error

Ejercicio 2.3 - Tabla de Multiplicar

Paso a paso:

1. Crear clase `TablaMultiplicar`

2. Solicitar número al usuario

3. Usar bucle for: `for (int i = 1; i <= 10; i++)`

4. Calcular y mostrar: `numero * i`

5. Formatear salida: "5 x 3 = 15"

Ejercicio 2.4 - Validación de Notas

Paso a paso:

1. Crear clase `ValidacionNotas`
2. Inicializar variables: suma = 0, contador = 0
3. Usar while loop:

```
java

while (true) {
    System.out.print("Ingrese nota (1-12) o -1 para terminar: ");
    int nota = scanner.nextInt();
    if (nota == -1) break;
    // validar rango y sumar
}
```

4. Calcular promedio fuera del loop
5. Mostrar resultado

Ejercicio 2.5 - Juego de Adivinanza

Paso a paso:

1. Crear clase `JuegoAdivinanza`
2. Generar número aleatorio: `int numeroSecreto = (int)(Math.random() * 100) + 1;`
3. Inicializar intentos = 0
4. Usar do-while para el juego:

```
java

do {
    // solicitar número
    // comparar y dar pistas
    intentos++;
} while (numeroUsuario != numeroSecreto);
```

5. Mostrar cantidad de intentos

DESAFÍO ESTRUCTURAS: Crear un menú interactivo que ofrezca: 1) Calcular factorial, 2) Serie Fibonacci, 3) Números primos en un rango, 4) Salir. El programa debe seguir ejecutándose hasta que el usuario elija salir.

3. ARREGLOS

Ejercicio 3.1 - Gestión de Notas

Paso a paso:

1. Crear clase `GestionNotas`
2. Declarar arreglo: `double[] notas = new double[5];`
3. Llenar arreglo con Scanner:

```
java
for (int i = 0; i < notas.length; i++) {
    System.out.print("Ingresa nota " + (i+1) + ": ");
    notas[i] = scanner.nextDouble();
}
```

4. Calcular promedio recorriendo el arreglo
5. Encontrar máximo y mínimo usando bucles
6. Contar aprobados (nota ≥ 6)

Ejercicio 3.2 - Lista de Compras

Paso a paso:

1. Crear clase `ListaCompras`
2. Declarar: `String[] productos = new String[8];`
3. Llenar arreglo con productos
4. Mostrar lista numerada con for
5. Implementar búsqueda:

```
java
for (String producto : productos) {
    if (producto.equals(productoBuscado)) {
        // encontrado
    }
}
```

Ejercicio 3.3 - Análisis de Temperaturas

Paso a paso:

1. Crear clase `AnalisisTemperaturas`
2. Declarar: `double[] temperaturas = new double[7];`

3. Llenar con temperaturas de la semana
4. Calcular promedio
5. Encontrar máximo y mínimo guardando también el índice (día)
6. Mostrar resultados con nombres de días

Ejercicio 3.4 - Ordenamiento de Números

Paso a paso:

1. Crear clase `OrdenamientoBurbuja`
2. Llenar arreglo de 10 números
3. Implementar burbuja:

```
java

for (int i = 0; i < numeros.length - 1; i++) {
    for (int j = 0; j < numeros.length - 1 - i; j++) {
        if (numeros[j] > numeros[j + 1]) {
            // intercambiar
        }
    }
}
```

4. Mostrar arreglo antes y después

Ejercicio 3.5 - Estadísticas de Ventas

Paso a paso:

1. Crear clase `EstadisticasVentas`
2. Declarar: `double[] ventas = new double[12];`
3. Llenar con datos de ventas mensuales
4. Calcular suma total
5. Encontrar mes con mayor y menor venta
6. Calcular promedio mensual
7. Mostrar reporte formateado con nombres de meses

DESAFÍO ARREGLOS: Crear un sistema de gestión de calificaciones para una clase de 20 estudiantes con 4 materias cada uno. Usar arreglo bidimensional, calcular promedio por estudiante, promedio por materia, y ranking de estudiantes.

4. OBJETOS DEL SISTEMA OPERATIVO

Ejercicio 4.1 - Información del Sistema

Paso a paso:

1. Crear clase `InformacionSistema`

2. Usar `System.getProperty()`:

```
java

String usuario = System.getProperty("user.name");
String os = System.getProperty("os.name");
String javaVersion = System.getProperty("java.version");
String directorioActual = System.getProperty("user.dir");
```

3. Mostrar información formateada

4. Agregar más propiedades interesantes

Ejercicio 4.2 - Medición de Tiempo

Paso a paso:

1. Crear clase `MedicionTiempo`

2. Obtener tiempo inicial: `long inicio = System.currentTimeMillis();`

3. Ejecutar operación a medir:

```
java

for (int i = 0; i < 1000000; i++) {
    // operación simple
}
```

4. Obtener tiempo final: `long fin = System.currentTimeMillis();`

5. Calcular diferencia: `long duracion = fin - inicio;`

6. Mostrar resultado en milisegundos

Ejercicio 4.3 - Manejo de Archivos Básico

Paso a paso:

1. Crear clase `ManejoArchivos`

2. Importar: `import java.io.File;`

3. Crear objeto File: `File archivo = new File("datos.txt");`

4. Verificar existencia: `if (archivo.exists())`

5. Mostrar información: tamaño, si es archivo, si se puede leer/escribir

6. Listar archivos del directorio actual

Ejercicio 4.4 - Variables de Entorno

Paso a paso:

1. Crear clase `VariablesEntorno`
2. Obtener variable específica: `String path = System.getenv("PATH");`
3. Obtener todas: `Map<String, String> env = System.getenv();`
4. Recorrer y mostrar variables importantes
5. Manejar variables que pueden no existir

Ejercicio 4.5 - Propiedades del Sistema

Paso a paso:

1. Crear clase `PropiedadesSistema`
2. Obtener propiedades: `Properties props = System.getProperties();`
3. Usar enhanced for loop para recorrer
4. Filtrar propiedades más relevantes
5. Formatear salida de manera organizada

DESAFÍO SISTEMA: Crear una aplicación que genere un reporte completo del sistema: información de hardware disponible a través de Java, espacio en disco, memoria, crear un archivo de log con timestamp, y medir el rendimiento de diferentes operaciones.

5. CLASES Y OBJETOS

Ejercicio 5.1 - Clase Estudiante

Paso a paso:

1. Crear archivo `Estudiante.java`
2. Definir clase y atributos privados:

```
java

public class Estudiante {
    private String nombre;
    private String cedula;
    private String carrera;
    private int semestre;
```

3. Crear constructor:

java

```
public Estudiante(String nombre, String cedula, String carrera, int semestre) {  
    this.nombre = nombre;  
    // asignar otros atributos  
}
```

4. Crear métodos get y set para cada atributo
5. Crear método `mostrarInfo()`
6. En clase principal, crear objetos y probar métodos

Ejercicio 5.2 - Clase Cuenta Bancaria

Paso a paso:

1. Crear clase `CuentaBancaria`
2. Atributos: numeroCuenta, titular, saldo
3. Constructor que inicialice todos los atributos
4. Método depositar():

java

```
public void depositar(double monto) {  
    if (monto > 0) {  
        saldo += monto;  
        System.out.println("Depósito exitoso");  
    }  
}
```

5. Método retirar() con validación de saldo
6. Métodos consultarSaldo() y mostrarDatos()
7. Crear clase de prueba

Ejercicio 5.3 - Clase Producto

Paso a paso:

1. Crear clase `Producto`
2. Atributos: codigo, nombre, precio, stock
3. Constructor y métodos get/set
4. Método calcularValorInventario(): `return precio * stock;`
5. Método verificarDisponibilidad(): `return stock > 0;`
6. Método aplicarDescuento() que reciba porcentaje

7. Crear varios productos y probar métodos

Ejercicio 5.4 - Clase Vehículo

Paso a paso:

1. Crear clase `Vehiculo`
2. Atributos: marca, modelo, año, kilometraje
3. Constructor completo
4. Método `acelerar()` que aumente kilometraje
5. Método `calcularAntiguedad()`:

```
java

public int calcularAntiguedad() {
    return 2025 - año;
}
```

6. Método `mostrarInformacion()`
7. Crear instancias y simular viajes

Ejercicio 5.5 - Sistema de Biblioteca

Paso a paso:

1. Crear clase `Libro`:
 - Atributos: titulo, autor, isbn, disponible
 - Constructor y métodos básicos
2. Crear clase `Biblioteca`:
 - Atributo: arreglo de libros
 - Constructor que inicialice arreglo
3. Método `agregarLibro()`:

```
java

public void agregarLibro(Libro libro) {
    // buscar posición libre y agregar
}
```

4. Método `buscarPorTitulo()` que retorne `Libro` o `null`
5. Métodos `prestarLibro()` y `devolverLibro()`
6. Crear clase principal para probar el sistema

DESAFÍO FINAL - CLASES Y OBJETOS: Crear un sistema completo de gestión escolar con las clases:

- **Estudiante** (nombre, cedula, notas por materia)
- **Profesor** (nombre, cedula, materias que enseña)
- **Materia** (nombre, código, profesor asignado)
- **Curso** (nombre, lista de estudiantes, lista de materias)
- **Escuela** (lista de cursos, métodos de gestión)

El sistema debe permitir:

- Inscribir estudiantes a cursos
 - Asignar profesores a materias
 - Registrar calificaciones
 - Generar reportes de promedio por curso
 - Listar estudiantes por profesor
 - Calcular ranking de estudiantes
-

DESAFÍO INTEGRADOR FINAL

Proyecto: Sistema de Gestión de Tienda

Crear un sistema completo que integre todos los conceptos vistos:

Requerimientos:

1. **Variables:** Manejar información de productos, ventas, clientes
2. **Estructuras de control:** Menú interactivo, validaciones, búsquedas
3. **Arreglos:** Inventario de productos, histórico de ventas
4. **Objetos del SO:** Generar reportes con timestamp, medir rendimiento
5. **Clases:** Producto, Cliente, Venta, Tienda

Funcionalidades mínimas:

- Registrar productos con código, nombre, precio, stock
- Registrar clientes con datos personales
- Realizar ventas (con validación de stock)
- Consultar inventario y generar reportes
- Buscar productos y clientes
- Calcular estadísticas de ventas
- Exportar datos a archivo de texto
- Interfaz de menú intuitiva

Criterios de evaluación:

- Funcionalidad completa (40%)
- Uso correcto de conceptos (30%)
- Calidad del código y documentación (20%)
- Creatividad e innovación (10%)

Tiempo estimado: 2 semanas

Profesor: [Nombre del docente]

DGETP - Programación 1

Año lectivo: 2025